Simultaneous Games p.16-28

Q. pure vs mixed strategy 차이?

Q. minimax theorem(max min=min max)?

Q. 동시게임 payoff matrix 읽기?

Single-move Simultaneous Game 정의 — Players {A,B}, 각자 동시에 action 선택. `V(a,b)` = A의 utility (B의 utility = −V, zero-sum).

- **Payoff matrix** V: 행=A의 action, 열=B의 action
- Two-finger Morra 예: V(1,1)=+2, V(1,2)=−3, V(2,1)=−3, V(2,2)=+4

A \ B	1 finger	2 fingers
1 finger	+2	−3
2 fingers	−3	+4

Two-finger Morra payoff matrix — A의 관점(양수=A 이득)

Strategy 정의

- **Pure strategy** — 단일 action `a ∈ Actions` (결정론적)
- **Mixed strategy** — 확률분포 `π` s.t. $0 \le \pi(a) \le 1$, $\Sigma\pi(a)=1$ (확률론적)
- Game value: $V(\pi_A, \pi_B) = \Sigma_{a,b}\ \pi_A(a)\cdot\pi_B(b)\cdot V(a,b)$

Pure strategy — 선공의 불이익 — A 선공: $\max_a \min_b V(a,b)$ ≤ B 선공: $\min_b \max_a V(a,b)$

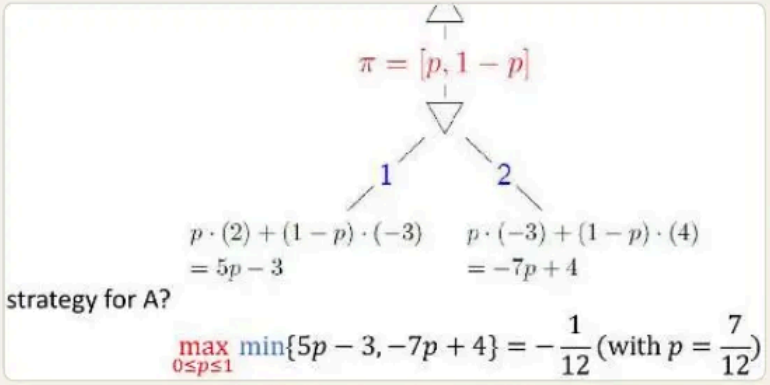
- 명제: **나중에 두는 것이 불리하지 않다** — 상대 전략을 보고 최적 대응 가능
- 이 부등식은 pure strategy에서 일반적으로 *등호가 성립하지 않는다*

Mixed strategy — second player는 pure로 충분 — 임의의 고정 mixed π_A 에 대해, $\min_{\pi_B} V(\pi_A,\pi_B)$ 는 pure strategy π_B 로 달성 가능.

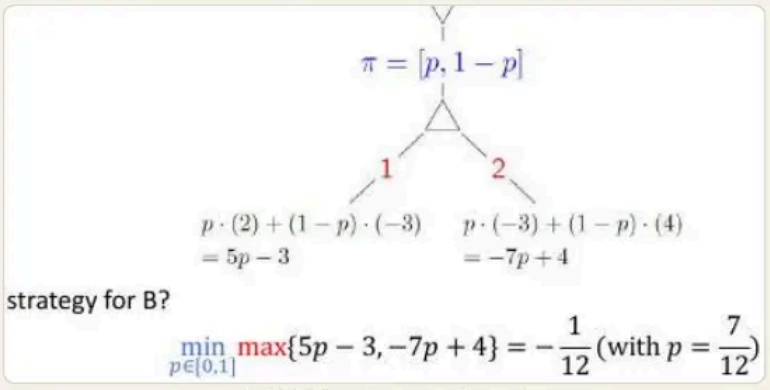
- 이유: $V(\pi_A, \pi_B)$는 π_B에 대해 **선형(linear/convex combination)** → 최솟값은 꼭짓점(pure)

Mixed strategy 최적 계산 — 과정 — Morra 예: A가 먼저 $\pi=[p,1-p]$ 를 공개. B는 pure 1 또는 2 선택.

- B가 1 선택 시 value = $p\cdot 2+(1-p)\cdot(-3) = 5p-3$
- B가 2 선택 시 value = $p\cdot(-3)+(1-p)\cdot 4 = -7p+4$



① A 먼저 공개 — max_p min{5p−3, −7p+4}



② B 먼저 공개 — min_p max{5p−3, −7p+4}

Mixed strategy game tree — A/B 순서 무관, 최적값 동일 (p.25-26)

Minimax 정리 (von Neumann 1928) — 유한 행동 2인 제로섬 게임에서 mixed strategy 범위로: max_π_A min_π_B V = min_π_B max_π_A V

- 최적 mixed strategy를 먼저 공개해도 손해 없음 — **전략 노출이 불이익이 되지 않는다**
- 두 순서 모두 동일한 game value (Morra: `−1/12`, p=7/12에서)
- 두 순서의 부등식이 등호로 성립: pure에서는 ≤, mixed에서는 =

> ★ **필수암기** minimax theorem: mixed strategy 시 max min = min max (등호). Pure strategy에서는 max min ≤ min max(부등호). Second player는 always pure strategy로 최적 달성 가능.

Non-zero-sum Games & Nash Equilibrium p.29-34

Q. Nash equilibrium 정의(단독이탈)?

Q. 존재정리 — 항상 존재?

Q. Pareto 최적과의 관계 (죄수딜레마)?

Utility 구조에 따른 게임 분류

- **Competitive (zero-sum)**: minimax로 풂 (linear programming)
- **Collaborative**: pure maximization (plain search)
- **Non-zero-sum**: 각 플레이어 utility `U_i(a,b)` 별도 — minimax 불성립

Prisoner's Dilemma — 대표 Non-zero-sum — A와 B 각각 testify/refuse 선택. 서로 다른 utility 구조.

- (testify, testify): U_A=−5, U_B=−5
- (testify, refuse): U_A=0, U_B=−10
- (refuse, testify): U_A=−10, U_B=0
- (refuse, refuse): U_A=−1, U_B=−1

A \ B	testify	refuse
testify	−5, −5	0, −10
refuse	−10, 0	−1, −1

Prisoner's Dilemma payoff (U_A, U_B) — Nash equilibrium = (testify, testify)

Nash Equilibrium 정의 — (π*_A, π*_B)가 Nash equlibrium ↔ 어느 플레이어도 단독으로 전략을 바꿀 인센티브가 없는 안정점

- `U_A(π*_A, π*_B) ≥ U_A(π_A, π*_B)` for all π_A
- `U_B(π*_A, π*_B) ≥ U_B(π*_A, π_B)` for all π_B
- Nash's existence theorem: 유한 플레이어·유한 행동 게임에서 Nash equilibrium이 반드시 존재한다

Nash Equilibrium 예시

- Two-finger Morra (zero-sum): A·B 모두 `π=[7/12, 5/12]` — minimax 해와 일치
- Collaborative Morra: Nash equilibrium 2개 — (둘다 1, value=2) / (둘다 2, value=4)
- Prisoner's Dilemma: (testify, testify) — 개인 이성이 집단 최적(refuse,refuse)을 달성 못 함
- Collaborative game: Nash eq = 행·열 내 다른 셀이 모두 더 작지 않은 항

⚠ **함정** Prisoner's dilemma: (refuse,refuse)이 Pareto 최적이나 Nash equilibrium이 아님. A가 단독으로 testify로 바꾸면 −10→0으로 이득. Nash eq = (testify,testify) — 개인합리성이 집단비효율을 낳는 전형적 트랩.

Zero-sum vs Non-zero-sum 비교

- Zero-sum: minimax theorem — unique game value, multiple minimax strategies
- Non-zero-sum: Nash theorem — multiple Nash equilibria, multiple game values 가능

	Zero-sum	Non-zero-sum
이론	von Neumann minimax (1928)	Nash existence (1950)
해법	Linear programming	Nash equilibrium 탐색
game value	유일(unique)	여러 개 가능
전략 공개	최적 mixed 공개 무해	공개 전략이 불이익될 수 있음

동시수행 게임 — zero-sum vs non-zero-sum 핵심 비교

★ **필수암기** 정리: TD learning w ← w−η[V(s)−(r+γV(s'))]∇V(s). Mixed minimax = max min = min max (pure에선 ≤). Nash eq = 누구도 단독이탈 불가인 안정점. Prisoner's dilemma Nash eq = (testify,testify), Pareto opt = (refuse,refuse).

요약 TD learning: **prediction−target=TD error**로 w 갱신(on-policy) · Q-learning은 max a'로 off-policy · zero-sum 동시게임은 **mixed strategy → minimax theorem(max min=min max)** · non-zero-sum은 **Nash equilibrium** 항상 존재(단 Pareto 최적은 미보장, 죄수의 딜레마).

Ch 11 Factor Graphs & CSP 1

변수-기반 모델 · Factor Graph 정의 · CSP 모델링

State-based vs Variable-based Models p.3-10

Q. state-based vs variable-based 차이?

Q. 변수기반이 유리한 문제 유형은?

State-based models 한계 — 순서(order)가 중요한 탐색·MDP는 state가 핵심. 그러나 순서가 무관한 문제에서는 비효율.

- takeaway 1: 지역 상호작용(local interactions)을 지정 → 전역 최적(global optimum) 탐색
- takeaway 2: state = 미래 최적 행동 선택에 충분한 과거 요약
- 순서가 무관(irrelevant)한 문제 → variable-based models가 더 적합

Variable-based models — 변수들에 값을 할당하는 *assignment* 문제. 대표: CSP, PGM(Markov/Bayesian networks, HMM, CRF).

- Modeling: 변수·도메인·팩터(제약) 지정
- Algorithm: 할당 탐색(exact / approximate)

✎ **핵심** 핵심 구분: state-based(순서 중요·MDP·탐색) vs variable-based(순서 무관·CSP·PGM). CSP는 variable-based의 all-or-nothing(0/1) 팩터 특수 사례.

Factor Graph p.13-19

Q. 변수·팩터 노드의 의미는?

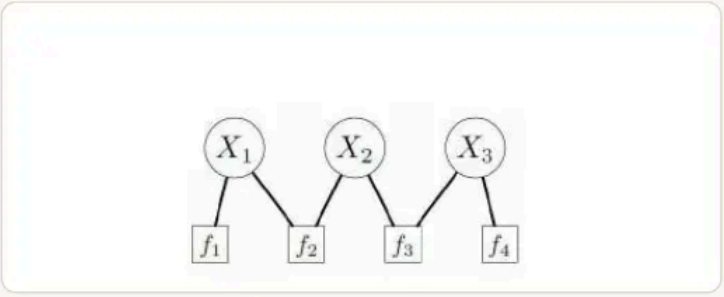
Q. Weight(x)=∏f ⱼ(x) 정의는?

Q. assignment의 weight 계산법?

Factor graph 정의

변수=원 팩터=사각형

- **Variables**: X = (X₁, ⋯, Xₙ), X_i ∈ Domain_i
- **Factors**: f₁, ⋯, fₘ, 각 f_j(X) ≥ 0 (비음수)
- 그래프 표현: 원(variable) + 사각형(factor) + 간선(scope 연결)



Factor graph 구조 — X₁,X₂,X₃와 f₁~f₄ (p.14)

Scope & Arity — factor f의 scope = 그 팩터가 의존하는 변수 집합. arity = scope의 크기.

- Unary factor: arity 1 (변수 1개에만 의존)
- Binary factor: arity 2 (변수 2개에 의존)
- **Constraint**: f ∈ {0,1} 인 팩터. 1=만족, 0=위반

Assignment Weight — 각 assignment x = (x₁,⋯,xₙ)의 weight = 모든 팩터값의 곱.

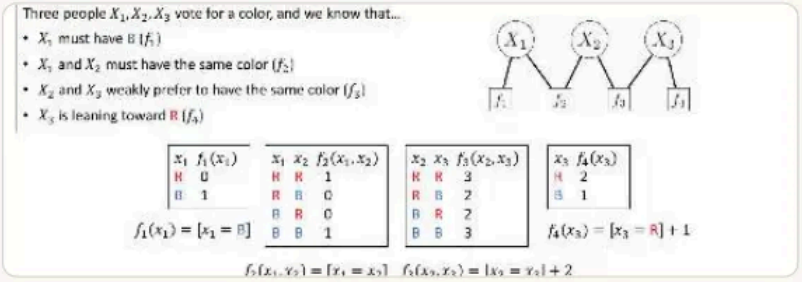
veto power: 하나라도 0이면 weight=0

- Weight(x) = ∏ⱼ f ⱼ(x) ← 곱, 합이 아님
- 어느 한 팩터라도 0이면 전체 weight = 0 (veto power)
- **목적**: arg max_x Weight(x) 를 찾는 것

> ⚠ **함정** Weight는 곱(product)이다. 더하기(sum)로 혼동 금지. Constraint factor가 하나라도 위반되면 전체 weight=0 → 자동 제거.

예제: 투표 (voting) — X₁,X₂,X₃ ∈ {R,B}. 4개 팩터(단항 2개·이항 2개). Weight 계산.

- f₁(x₁)=[x₁=B], f₂(x₁,x₂)=[x₁=x₂], f₃(x₂,x₃)=[x₂=x₃]+2, f₄(x₃)=[x₃=R]+1
- 최대 weight 할당: (B,B,R) ⟶ Weight = 1·1·2·2 = 4



투표 예제 — 팩터 테이블과 전체 weight 계산 (p.13)

Constraint Satisfaction Problem (CSP) p.20-21

Q. CSP가 factor graph의 특수사례인 이유?

Q. consistent assignment 조건은?

Q. constraint의 veto power란?

CSP 정의 — 모든 팩터가 constraint(0 or 1)인 factor graph 특수 사례.

- f ⱼ(x) ∈ {0,1} for all j
- **Consistent assignment**: Weight(x) = 1, 즉 모든 constraint 만족
- **목적**: consistent assignment 존재 여부 판별 (또는 탐색)

	Factor graph (일반)	CSP (특수)
단위	변수(variable)	변수(variable)
평가 함수	팩터 f_j(x) ≥ 0	제약 f_j(x) ∈ {0,1}
목적	max Weight(x)	consistent assignment 탐색
weight=0 의미	나쁜 할당	제약 위반 (불가)

Factor graph vs CSP 비교

CSP 모델링 p.22-28

Q. 변수·도메인·제약 설계 절차?

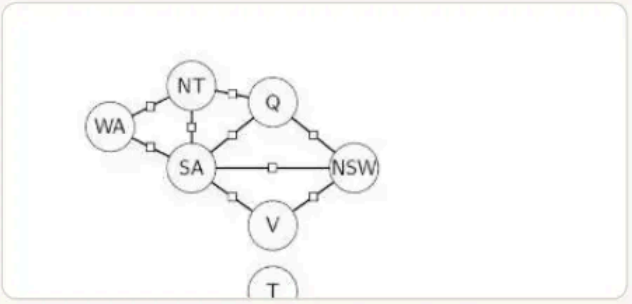
Q. 좋은 모델링의 기준은?

모델링 절차 — 문제 → CSP로 변환하는 4단계.

- ① **변수·도메인 결정**: 무엇을 모를지 변수로, 도메인은 가능한 값 집합
- ② **팩터(제약) 설계**: 각 요구사항을 constraint로 변환
- ③ **크기 최소화**: 변수·팩터·도메인·arity를 작게 유지
- ④ **검증 관점**: 해를 구하는 게 아니라 주어진 해가 맞는지 체크하는 방향으로 팩터 구현

예제: Map Coloring (호주 7개 주) — 7개 주를 {R,G,B}로 칠하되, 인접 주는 다른 색.

- Variables: X = (WA, NT, SA, Q, NSW, V, T), Domain_i = {R,G,B}
- Factors(constraints): [WA≠NT], [NT≠Q], [NT≠SA], [SA≠Q], [SA≠NSW], [SA≠V], [Q≠NSW], [NSW≠V]
- Weight([각 인접쌍 다른색]) = 1 ← consistent
- Tasmania(T)는 **독립(isolated)** → 다른 변수에 영향 없음



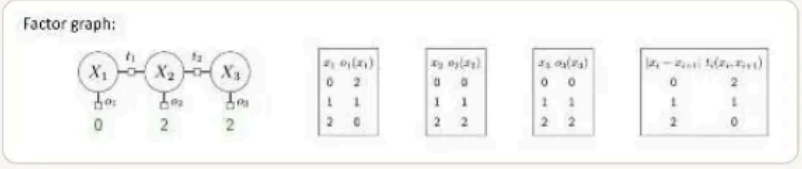
Map coloring constraint graph (p.15)

예제: LSAT (Art Gallery) — 조각품 A,B,C → 방 1,2에 배치. 3개 조건.

- C1: A,B 같은 방 불가 [room(A)≠room(B)]
- C2: B,C 같은 방 [room(B)=room(C)]
- C3: 방 2는 하나만 [Σ{i:room(i)=2} 1 ≤ 1]
- C1∧C2: A≠B, B=C ⟹ A≠C. C3: 방2에 1개만 ⟹ 해: A→방2, B,C→방1 (또는 A→방1, B,C→방2이면 C3 위반)

예제: Object Tracking — 센서 관측 0,2,2 → 물체 실제 경로 추정. CSP + 연속 시간 구조.

- Variables: X_i ∈ {0,1,2} — 시각 i의 위치
- Observation factors o_i(x_i): 센서값과의 호환성 (노이즈 허용)
- Transition factors t_i(x_i,x_{i+1}): |x_i − x_{i+1}| ≤ 1 (텔레포트 금지)
- Factor graph = 체인(chain) 구조 → **순서 있는 변수** 간 국소 제약



Object tracking factor graph — 체인 구조 + 팩터 테이블 (p.23)

예제: Event Scheduling (두 가지 정식화) — E개 이벤트, T개 시간슬롯. C1/C2/C3 만족.

- Formulation 1 — 변수: X_e ∈ {1,⋯,T} (이벤트→슬롯). C2: 모든 e≠e′에 대해 [X_e≠X_{e′}] → 팩터 수 $O(E^2)$. C3: [(e,X_e)∈A]
- Formulation 2 — 변수: Y_t ∈ {1,⋯,E}∪{∅} (슬롯→이벤트). C1: 각 e가 정확히 하나의 t에 배정. C3: [(Y_t=∅) or (Y_t,t)∈A]
- 같은 문제라도 변수 정의 방향에 따라 팩터 구조·수가 달라짐

예제: Program Verification — def foo(x,y): a=x²; b=a+y²; c=b-2xy; return c. Spec: c≥0 for all x,y.

- Variables: x, y, a, b, c
- Constraints (프로그램 구문): [a=x²], [b=a+y²], [c=b-2xy]
- Constraint (명세 부정): [c<0]
- CSP에 consistent assignment 없으면 → 프로그램이 명세 만족 (반례 없음)
- CSP에 consistent assignment 있으면 → 반례 존재 (버그)

✎ 핵심 Program verification의 CSP 핵심: 명세의 부정(negation)을 constraint로 추가. CSP UNSAT ⟺ 프로그램 정확. 프로그램 assignment(=) → CSP equality(=)로 변환.

요약 & 필수암기 p.28

Q. factor graph ↔ CSP 핵심 한 줄?

CSP 모델링 체크리스트

- 변수·도메인 결정 → 팩터(constraint) 설계 → 크기 최소화 → 검증 관점으로 구현
- Factor graph Weight = $\prod f_j(x)$ (곱), CSP는 0/1 팩터만
- Consistent ⟺ Weight=1 ⟺ 모든 constraint 만족
- 같은 문제에 formulation이 여러 개 가능 → 팩터 수·복잡도 다름

★ 필수암기 Factor graph: Variables X_i∈Domain_i, Factors f_j≥0, Weight(x)=∏f_j(x), 목적=arg max_x Weight(x). CSP: 팩터 all 0/1, consistent=Weight=1. Scope=factor가 의존하는 변수집합, Arity=scope크기. Veto power: 한 팩터=0이면 전체 weight=0.

요약 Factor graph = 변수(원)+팩터(사각형) 이분 그래프, Weight(x)=$\prod f_j(x)$ 최대화 · CSP = 팩터가 모두 0/1인 특수 사례(consistent=Weight 1) · 모델링 핵심=변수·제약 설계 + veto power.

Ch 12 Factor Graphs & CSP 2

Backtracking · Arc Consistency · Beam Search · Local Search (ICM)

Factor Graph & Assignment 기본 정의 p.3-6

Q. partial vs complete assignment?

Q. weight·consistency 정의는?

Factor graph — 변수 $X=(X_1,\cdots,X_n)$ + factor $f_1,\cdots,f_m$ (각 $f_j\geq 0$). 목표: max Weight(x) = $\max_x \prod_j f_j(x_scope(j))$

- Assignment weight = $\prod_j f_j(x)$ — 모든 factor의 곱
- Partial assignment a: 일부 변수에만 값 배정. weight = *scope가 배정된 변수만인 factor들의 곱*

Dependent factors — 변수 X_i 배정 시 활성화되는 factor 집합. $dep(a, X_i) = \{ f_j : scope(f_j) \subseteq assigned(a) \cup \{X_i\} \}$

- 예: a={WA:R, NT:G}, X_i=SA → dep = {[WA≠SA], [NT≠SA]}
- 배정 직후 즉시 계산 가능한 partial weight 인수들